

LAB PRACTICUM: OpenMP Multi-Core Scaling in Python

Hands-Off Benchmark Practicum • 60 Minutes • Autonomous Execution

Student Name:	Abyz Nuradil	Student ID:	230103188
Hardware / CPU:	AMD EPYC 9V74 (workspace)	Date & Section:	01.10.2026 / section not supplied

Benchmark provenance: automated Linux workspace; 9 threads, CPU quota 8 cores.

LAB PROTOCOL & AUTONOMOUS RULES OF ENGAGEMENT

1. Autonomous Session: You are expected to read, run, optimize, and document all findings independently.
2. Zero Support for Dependency Issues: If you encounter an error, verify that you installed packages inside a clean virtual environment. Compilers are NOT needed for this lab.
3. Equal Hardware Standard: Keep your laptop plugged into wall power and close background browsers, video streams, or games to prevent CPU thermal throttling and frequency capping.
4. Whiteboard Leaderboard: When you finish Task 1 and Task 3, record your CPU model and best speedup factor on the lab whiteboard.

1. Background & Environment Setup (10 Minutes)

Standard CPython executes bytecode under the Global Interpreter Lock (GIL), preventing multiple CPU threads from executing simultaneously in Python space. In this lab, we bypass the GIL using Numba's JIT compiler with its native multithreaded OpenMP/TBB work-stealing engine. Because Numba compiles numeric Python loops directly to machine code and releases the GIL inside parallel loops (prange), we achieve bare-metal native CPU scaling across both macOS (Apple Silicon M-series) and Windows/Linux x86_64 machines with zero local C++ compiler dependencies.

Execute the following in your terminal / command prompt:

```
pip install numpy numba matplotlib
```

2. Challenge 1: The Core Speedometer & Amdahl's Law

Objective: Benchmark an embarrassingly parallel Monte Carlo simulation to approximate π across varying thread counts. You will measure execution latency, quantify parallel speedup, and observe where Amdahl's Law caps performance.

```
import time
import numpy as np
import numba
from numba import njit, prange

print(f"Hardware Threads Detected: {numba.config.NUMBA_NUM_THREADS}")

@njit(parallel=True)
def monte_carlo_pi(n_samples):
    inside_circle = 0
    for i in prange(n_samples):
        x = np.random.uniform(0.0, 1.0)
        y = np.random.uniform(0.0, 1.0)
        if x * x + y * y <= 1.0:
            inside_circle += 1
    return (4.0 * inside_circle) / n_samples

# JIT Warmup (excludes JIT compilation overhead from benchmarks)
_ = monte_carlo_pi(10_000)

SAMPLES = 120_000_000
thread_counts = [1, 2, 4, 8, numba.config.NUMBA_NUM_THREADS]
# Filter out duplicates and invalid counts
thread_counts = sorted(list(set([t for t in thread_counts if t <=
numba.config.NUMBA_NUM_THREADS])))

print(f"{'Threads':<10} | {'Time (s)':<12} | {'Speedup':<10} | {'Efficiency'
('%)':<15}")
print("-" * 55)

t1_baseline = None
for t in thread_counts:
    numba.set_num_threads(t)
    start = time.perf_counter()
    pi_est = monte_carlo_pi(SAMPLES)
    elapsed = time.perf_counter() - start

    if t == 1:
        t1_baseline = elapsed
        speedup = 1.0
        efficiency = 100.0
    else:
```

```
speedup = t1_baseline / elapsed
efficiency = (speedup / t) * 100.0

print(f"{t:<10} | {elapsed:<12.4f} | {speedup:<10.2f}x |
{efficiency:<15.1f}")
```

Deliverables & Technical Questions for Challenge 1:

- A. Record your baseline T_1 (1 thread) and your maximum multi-thread runtime $T_{\max}$ in Table 1 below.
- B. Why is the warm-up call `_ = monte_carlo_pi(10_000)` strictly required before recording benchmark timings?
- C. Did your parallel efficiency stay at 100% when scaling from 1 to max threads? What architectural factors (e.g. memory bus contention, hyperthreading/SMT core vs physical core limits) explain the degradation?
- D. Why does `inside_circle += 1` inside `prange` not corrupt the count? What implicit OpenMP reduction construct does Numba generate here?

3. Challenge 2: Load Imbalance & Dynamic Scheduling

Objective: Compute a high-resolution 2500×2500 Mandelbrot fractal. Unlike Monte Carlo, points in the complex plane require radically different amounts of compute: points inside the set iterate all the way to `max_iter`, while outer points escape in 2-3 iterations. You will investigate how non-uniform workloads impact thread utilization.

```
import time
import numpy as np
import matplotlib.pyplot as plt
from numba import njit, prange

@njit(parallel=True)
def render_mandelbrot_rows(h, w, max_iter):
    img = np.zeros((h, w), dtype=np.int32)
    # Row-parallel loop
    for r in prange(h):
        cy = -1.2 + (r / h) * 2.4
        for c in range(w):
            cx = -2.0 + (c / w) * 2.5
            z_real, z_imag = 0.0, 0.0
            it = 0
            while (z_real * z_real + z_imag * z_imag <= 4.0) and (it <
max_iter):
                next_real = z_real * z_real - z_imag * z_imag + cx
                z_imag = 2.0 * z_real * z_imag + cy
                z_real = next_real
                it += 1
            img[r, c] = it
    return img

@njit(parallel=True)
def render_mandelbrot_cols(h, w, max_iter):
    img = np.zeros((h, w), dtype=np.int32)
    # Column-parallel loop (swapped iteration axis)
    for c in prange(w):
        cx = -2.0 + (c / w) * 2.5
        for r in range(h):
            cy = -1.2 + (r / h) * 2.4
            z_real, z_imag = 0.0, 0.0
            it = 0
            while (z_real * z_real + z_imag * z_imag <= 4.0) and (it <
max_iter):
                next_real = z_real * z_real - z_imag * z_imag + cx
                z_imag = 2.0 * z_real * z_imag + cy
                z_real = next_real
                it += 1
```

```

        img[r, c] = it
    return img

# Warmup JIT
_ = render_mandelbrot_rows(100, 100, 50)
_ = render_mandelbrot_cols(100, 100, 50)

H, W, MAX_IT = 2500, 2500, 1000

t0 = time.perf_counter()
grid_rows = render_mandelbrot_rows(H, W, MAX_IT)
t_rows = time.perf_counter() - t0

t1 = time.perf_counter()
grid_cols = render_mandelbrot_cols(H, W, MAX_IT)
t_cols = time.perf_counter() - t1

print(f"Row-Parallel Render Time:    {t_rows:.3f} s")
print(f"Column-Parallel Render Time: {t_cols:.3f} s")

# Save visual artifact
plt.figure(figsize=(8, 8))
plt.imshow(grid_rows, cmap='magma', extent=[-2.0, 0.5, -1.2, 1.2])
plt.title(f"Mandelbrot {H}x{W} (Render: {t_rows:.2f}s)")
plt.axis('off')
plt.savefig('mandelbrot_output.png', dpi=300, bbox_inches='tight')
print("Saved image: mandelbrot_output.png")

```

Deliverables & Technical Questions for Challenge 2:

- A. Compare `t_rows` vs `t_cols`. Which axis decomposition completed faster on your machine and why?
- B. Explain how NumPy's C-contiguous (row-major) memory layout interacts with CPU cache lines when threads write to `img[r, c]` vs `img[c, r]`.
- C. Load imbalance analysis: If thread 0 receives the top 20% of rows and thread 2 receives the center 20% of rows, why does one thread idle while the other struggles? How does OpenMP dynamic scheduling resolve this?
- D. Attach your exported `mandelbrot_output.png` file alongside your completed task sheet.

4. Challenge 3: Stencil Computation & Memory Bandwidth

Objective: Simulate 2D heat diffusion using a 5-point discrete Laplacian stencil over 300 timesteps on a 1500×1500 grid. Unlike the compute-bound fractal, stencil computations are strictly memory-bandwidth bound. You will determine whether adding CPU cores continues to speed up performance or hits the RAM memory bus wall.

```
import time
import numpy as np
from numba import njit, prange

@njit(parallel=True)
def heat_step(u, u_next, alpha=0.20):
    rows, cols = u.shape
    for i in prange(1, rows - 1):
        for j in range(1, cols - 1):
            u_next[i, j] = u[i, j] + alpha * (
                u[i+1, j] + u[i-1, j] + u[i, j+1] + u[i, j-1] - 4.0 * u[i, j]
            )

GRID_SIZE = 1500
STEPS = 300

u = np.zeros((GRID_SIZE, GRID_SIZE), dtype=np.float64)
u_next = np.zeros_like(u)

# Dirichlet Boundary conditions: Top & Left walls maintained at 100°C
u[0, :] = 100.0
u[:, 0] = 100.0
u_next[0, :] = 100.0
u_next[:, 0] = 100.0

# Warmup
heat_step(u, u_next)

# Benchmark Execution
start = time.perf_counter()
for step in range(STEPS):
    heat_step(u, u_next)
    # Pointer swap (avoids array allocation copies)
    u, u_next = u_next, u
elapsed = time.perf_counter() - start

cells_per_sec = (GRID_SIZE * GRID_SIZE * STEPS) / elapsed / 1e6
print(f"Heat Diffusion Complete: {elapsed:.3f} s")
print(f"Throughput: {cells_per_sec:.2f} Megacells/sec")
```

Deliverables & Technical Questions for Challenge 3:

- A. Record your benchmark runtime and Megacells/sec throughput in Table 1.
- B. Change the array precision from `np.float64` to `np.float32` . Re-run the test. By what factor did runtime decrease? Why does halving memory footprint improve stencil throughput far more than compute capability?
- C. Memory Roofline Model: Explain why adding twice as many CPU cores does not produce a 2x speedup on stencil codes once memory bandwidth is saturated.

5. Student Result Scorecard & Data Collection

Fill in your benchmark results into Table 1 below before submitting this document:

Benchmark Challenge	Active Threads	Array Size / Samples	Time (seconds)	Speedup Factor	Efficiency (%)
Challenge 1: Monte Carlo	1 Thread (Base)	120,000,000	2.180438	1.00x	100.0%
Challenge 1: Monte Carlo	2 Threads	120,000,000	1.451848	1.502x	75.09%
Challenge 1: Monte Carlo	4 Threads	120,000,000	0.777060	2.806x	70.15%
Challenge 1: Monte Carlo	Max Threads (9 active threads)	120,000,000	0.377836	5.771x	64.12%
Challenge 2: Mandelbrot (Rows)	Max Threads (9 active threads)	2500 x 2500	1.272772	N/A	N/A
Challenge 2: Mandelbrot (Cols)	Max Threads (9 active threads)	2500 x 2500	1.300855	N/A	N/A
Challenge 3: Heat Stencil	Max Threads (9 active threads)	1500 x 1500 x 300	0.082922	N/A	N/A

Heat throughput (float64): 8140.18 Megacells/sec

Float32 comparison: 0.067184 s; 10047.05 Megacells/sec

Values are medians of three repetitions after JIT warmup. Full tables and answers follow.

CPU: AMD EPYC 9V74, Linux workspace. These results do not measure the student laptop.

OpenMP Multi-Core Scaling in Python - completed practicum

Student: Abyz Nuradil

Student ID: 230103188

Lab date: 01.10.2026

Section: Not supplied in the assignment request.

Benchmark environment: Automated Linux workspace; not the student laptop

Actual run timestamp (UTC): 2026-10-01T09:53:52.013087+00:00

Mode: full assigned workloads

Hardware and measurement method

CPU: **AMD EPYC 9V74 80-Core Processor**. Visible logical CPUs: 9; Numba maximum: **9 threads**; container CPU quota: 8 core equivalents. Backend: **omp**. OS: Linux-6.18.44-x86_64-with-glibc2.39. Python 3.12.14; NumPy 2.5.3; Numba 0.68.0; Matplotlib 3.11.2.

Every result is the median of 3 measured repetitions. Each compiled kernel and both heat precisions are warmed up before timing. Heat grids are reset outside each timed region, so every run starts from identical initial conditions. Heat timing includes the Python timestep loop and pointer swaps; Mandelbrot timing includes image allocation. Raw repetition timings and all Monte Carlo estimates are preserved in `benchmark_results.json`.

These measurements describe the environment named above. They do not measure a different laptop. No claim of measured peak RAM bandwidth, SMT effects, or physical-core topology is made from these timings alone.

Table 1 - Student result scorecard

Benchmark	Active threads	Size / samples	Time (s)	Speedup	Efficiency (%)
Challenge 1: Monte Carlo	1	120,000,000	2.180438	1.000x	100.00
Challenge 1: Monte Carlo	2	120,000,000	1.451848	1.502x	75.09
Challenge 1: Monte Carlo	4	120,000,000	0.777060	2.806x	70.15
Challenge 1: Monte Carlo	8	120,000,000	0.418336	5.212x	65.15
Challenge 1: Monte Carlo	9	120,000,000	0.377836	5.771x	64.12
Challenge 2: Mandelbrot (rows)	9	2500 x 2500	1.272772	N/A	N/A
Challenge 2: Mandelbrot (cols)	9	2500 x 2500	1.300855	N/A	N/A
Challenge 3: Heat (float64)	9	1500 x 1500 x 300	0.082922	N/A	N/A
Challenge 3: Heat (float32)	9	1500 x 1500 x 300	0.067184	N/A	N/A

Challenge 1 - answers A-D

A. Baseline and maximum-thread runtime. $T_1 = 2.180438$ s. $T_{\max} = 0.377836$ s at 9 threads. Maximum-thread speedup is **5.771x**, with **64.12%** efficiency. The best observed speedup is **5.771x** at 9 threads. $T_{\max}$ means runtime at the maximum worker count, not the largest elapsed time.

B. Why warmup is required. The first invocation compiles Python into native code, specializes the argument types, and initializes parallel runtime resources. Including that work would compare compilation/startup cost with already compiled executions. The 10,000-sample warmup excludes JIT compilation from the measurements.

C. Parallel efficiency and Amdahl's Law. Efficiency decreased below 100% at the maximum worker count: **64.12%**. Speedup $S = T_1 / T_p$ and efficiency $E = 100 S / p$. Amdahl's Law is $S(p) = 1 / (s + (1-s)/p)$, where s is the serial fraction. Worker coordination, reductions and serial launch overhead limit scaling. CPU quotas and shared execution resources also matter here. On other hardware, memory/cache contention, thermal/frequency changes, and SMT threads sharing physical cores can contribute. Monte Carlo is largely compute-bound, so memory bandwidth is not automatically the dominant explanation. Timing alone does not identify which architectural factor caused a particular loss of efficiency.

D. Why the count is safe. Numba recognizes `inside_circle += 1` as an integer sum reduction. Each worker accumulates a private partial count, and those counts are combined after the parallel loop. With OpenMP selected, this has the semantics of `reduction(+:inside_circle)`; workers do not race on an ordinary shared increment. Numba can lower this to private accumulators and a combine phase rather than literal generated C source.

Challenge 2 - answers A-D

A. Measured comparison. Row-parallel time = **1.272772** s; column-parallel time = **1.300855** s. The **rows** decomposition was **1.022x** faster in this run. Both output arrays are exactly equal. Changing the decomposition changes both memory access and how expensive pixels are distributed. Contiguous row writes generally favor locality, but workload imbalance can offset that advantage. This run does not separately instrument cache misses or per-thread idle time, so the timing difference is consistent with those factors rather than proof of a single cause.

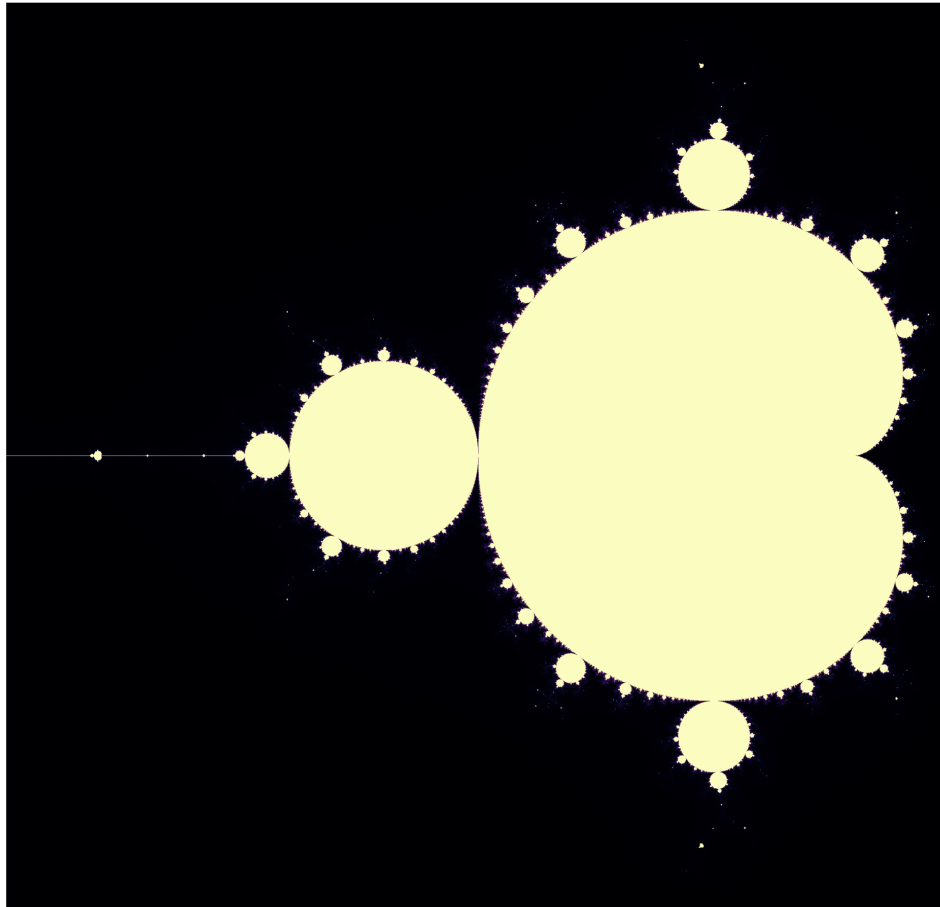
B. C-contiguous layout. A C-order int32 array stores adjacent columns in the same row at consecutive 4-byte addresses. The row kernel varies `c` inside the inner loop, writing `img[r, c]` contiguously. The column kernel varies `r` inside the inner loop, also writing `img[r, c]`, but consecutive writes are separated by `w * 4` bytes. At width 2500, that stride is 10,000 bytes. Strided writes use cache lines less effectively and can increase cache/TLB pressure. The PDF's `img[c, r]` example illustrates swapped indexing; our actual kernels retain the same coordinate mapping and array indexing.

C. Load imbalance and dynamic scheduling. An outer band may escape after a few iterations, while central points reach `max_iter`. Equal row counts therefore need not mean equal computation. With static contiguous chunks, a worker assigned cheap rows can finish while a worker assigned central rows continues. OpenMP `schedule(dynamic, chunk)` assigns another chunk to each available worker, reducing idle time at the cost of scheduling overhead. A row/column swap by itself does not enable dynamic scheduling. Numba's default prange scheduling is static; its positive chunk-size dynamic scheduling is supported by the TBB backend, so changing the chunk size with an OpenMP backend would not demonstrate dynamic scheduling. The recorded backend is **omp**, and these kernels retain default scheduling. See the [official Numba scheduling documentation](#).

D. Visual artifact. `mandelbrot_output.png` is exported at 300 DPI and accompanies this report and the completed task sheet. The lower image origin matches the coordinates used by the kernel.

Mandelbrot visual artifact

Mandelbrot 2500x2500 (Rows: 1.27s)



Challenge 3 - answers A-C

A. Runtime and throughput. At 9 threads, float64 ran in **0.082922 s**, producing **8140.18 Megacells/s**. Throughput follows the lab formula $\text{grid_size}^2 * \text{steps} / \text{seconds} / 1e6$. Only $(\text{grid_size}-2)^2$ interior cells are updated at each step; using the full grid is the worksheet's reporting convention.

B. Float32 comparison. At the same 9 threads, float32 ran in **0.067184 s**, producing **10047.05 Megacells/s**. The runtime ratio $T_{\text{float64}} / T_{\text{float32}} = 1.234$; runtime decreased in this measurement. The two buffers occupy **36.00 MB** in float64 and **18.00 MB** in float32. Halving stored element size can reduce data movement and fit more cells in cache. A 2x speedup is not guaranteed because cache reuse, arithmetic, dispatch/synchronization, and CPU quotas also affect total time. The maximum final-state precision difference is **0.00001541 C**, within the checked tolerance. $\alpha=0.20$ satisfies the explicit 2D stability limit $\alpha \leq 0.25$. Top and left boundaries remain 100 C; bottom and right remain 0 C except their intersections with the heated walls.

C. Memory roofline. The roofline bound is $P \leq \min(P_{\text{peak}}, B * I)$, where B is memory bandwidth and I is arithmetic intensity (operations per byte moved). This stencil performs relatively little arithmetic for its array traffic. Once the relevant memory bandwidth is saturated, doubling cores cannot double available bytes per second. Cache reuse can shift the relevant bottleneck toward cache bandwidth, and repeated parallel launches add overhead. The scaling tables below show observed runtimes; identifying RAM saturation specifically would require bandwidth or hardware-counter measurements.

Float64 scaling

Threads	Time (s)	Speedup	Efficiency (%)	Throughput (Mcells/s)
1	0.473171	1.000x	100.00	1426.54
2	0.240758	1.965x	98.27	2803.65
4	0.107021	4.421x	110.53	6307.20
8	0.087997	5.377x	67.21	7670.71
9	0.082922	5.706x	63.40	8140.18

Float32 scaling

Threads	Time (s)	Speedup	Efficiency (%)	Throughput (Mcells/s)
1	0.234506	1.000x	100.00	2878.39
2	0.152896	1.534x	76.69	4414.75
4	0.112266	2.089x	52.22	6012.48
8	0.074270	3.157x	39.47	9088.50
9	0.067184	3.491x	38.78	10047.05

Validation and leaderboard data

Full-resolution row and column fractals match exactly. Heat grids match exactly across thread counts within each precision; heated and cold boundaries stay fixed, and temperatures remain finite and between 0 and 100 C. Float32 and float64 states agree within the configured tolerance. Monte Carlo estimates are checked against pi using a stochastic tolerance.

Leaderboard entry for this benchmark environment: **AMD EPYC 9V74 80-Core Processor**, Monte Carlo best speedup **5.771x** (9 threads). Best float64 heat speedup: **5.706x**. Record laptop results separately after running on that laptop.